SYMPRAXIS CONSULTING

Deep Dive into Azure Function Bindings: Blob, Queues, Event Grid & More

Julie Turner

SYMPRAXIS CONSULTING



julieturner.net/me

Julie Turner

Partner / CTO · Sympraxis Consulting

- Working with SharePoint since **2007**
- **Microsoft MVP**, Office Apps & Services since 2017
- M365 & Power Platform **Community Team** since 2019
- Open-source co-maintainer: **PnPjs** & **hTWOo**
- Co-host: ***Ask Sympraxis***

Agenda

- Binding Fundamentals
- Extension Model & Connections
- Binding Demos
  - Blob Storage
  - Queue Storage
  - Event Grid
  - Azure SQL

What Is a Binding?

- A declarative connection between your function code and an Azure service
- Three directions:
  - Trigger — what starts the function (exactly one per function)
  - Input — read additional data into your function before processing
  - Output — write data out to another service after processing
- What bindings do for you:
  - Manage connection lifecycle and authentication
  - Handle serialization / deserialization of data
  - Provide built-in retry and error handling
- Your code just consumes or produces the data — no boilerplate
- Bindings are delivered as part of the default @azure/functions package

Why Input/Output Bindings

- **Input binding**: The runtime *fetches data for you* from an external service and passes it into your function as a parameter at invocation time.
- **Output binding**: The runtime *sends data for you* to an external service when your function completes.
- The key mental model: bindings are **declarative data pipelines**.
  - The trigger says *what wakes your function up*.
  - Input bindings say *what data to hand it when it wakes*.
  - Output bindings say *where to deliver what it produces*.
- The trade-off versus using the SDK directly
  - bindings are simpler and less code, but you lose fine-grained control.
  - SDK and Bindings can coexist in the same function.

Binding Anatomy

```
import { app, input, output } from '@azure/functions';

// 1. Declare an input binding
const myInput = input.storageBlob({
    path: 'container/{name}',
    connection: 'AzureWebJobsStorage'
});

// 2. Declare an output binding
const myOutput = output.storageQueue({
    queueName: 'results',
    connection: 'AzureWebJobsStorage'
});

// 3. Register the function — wire trigger + input + output
app.storageBlob('myFunction', {
    path: 'uploads/{name}',
    connection: 'AzureWebJobsStorage',
    extraInputs:  [myInput],
    extraOutputs: [myOutput],
    handler: async (blob, context) => {
        // Read from input binding
        const data = context.extraInputs.get(myInput);
        // Write to output binding
        context.extraOutputs.set(myOutput, 'processed result');
    }
});
```

Bindings available out of the box

Binding	Supports	Description
`app.http()`	`TO`	HTTP bindings - plus shorthand methods: app.get(), app.post(), app.put(), app.patch(), app.deleteRequest()
`app.timer()`	`T`	cron-scheduled execution
`app.storageBlob()`	`TIO`	triggers on blob container changes
`app.storageQueue()`	`TO`	triggers on queue messages
`app.serviceBusQueue()`	`TO`	triggers on Service Bus queue messages
`app.serviceBusTopic()`	`TO`	triggers on Service Bus topic subscriptions
`app.eventHub()`	`TO`	triggers on Event Hubs messages
`app.eventGrid()`	`TO`	triggers on Event Grid events
`app.cosmosDB()`	`TIO`	triggers on Cosmos DB change feed
`app.sql()`	`TIO`	triggers on Azure SQL row changes (create/update/delete)
`app.mySql()`	`TIO`	triggers on MySQL row changes (create/update only; preview)

Bindings available out of the box (continued)

Binding	Supports	Description
`app.warmup()`	`T`	fires during scale-out pre-warming (Premium plan only)
`app.webPubSub()`	`TIO`	triggers on Azure Web PubSub events
`app.connectorTrigger()`	`T`	triggers from Azure Logic Apps connector events
`app.mcpTool()`	`T`	exposes function as an MCP tool callable by AI agents
`app.mcpResource()`	`T`	exposes function as a readable MCP resource
`app.mcpPrompt()`	`T`	exposes function as an MCP prompt template
`app.generic()`		escape hatch for any trigger type not listed above
`table()`	`IO`	exposes access to table storage for input and output bindings

“durable-functions” package — Durable Functions (orchestrations, activities, entities, sagas).

Uses its own df.orchestration(), df.activity(), df.entity() registration pattern alongside @azure/functions

Connection “Better” Practices

- App Settings only - never put connection strings in source code
- local.settings.json for local dev (excluded from source control)
- Managed Identity is the modern recommended approach - No credentials to rotate or accidentally expose in code

Hosts.json - Setup

```
"functionTimeout": 30,
"extensionBundle": {
    "id": "Microsoft.Azure.Functions.ExtensionBundle",
    "version": "[4.0.0, 5.0.0)"
}
```

- functionTimeout default is 30 mins. (Consumption (legacy) was 5 mins)
- extensionBundle is required for bindings of any kind to work.
- The version is related to the version of AzureFunctions you're running.

Host.json - logging

```
"logging": {
  "applicationInsights": {
    "samplingSettings": {
      "isEnabled": true,
      "excludedTypes": "Request"
    }
  }
}
```

- By default, sampling is on and capped at 20 telemetry items/sec per instance.
- High-throughput queues will silently drop execution telemetry.
- excludedTypes are those that should not be sampled, Requests are the invocation of the trigger.

Blob Storage Bindings



SYMPRAXIS CONSULTING

Blob Storage: hosts.json configuration

```
"extensions": {
    "blobs": {
        "maxDegreeOfParallelism": 4,
        "poisonBlobThreshold": 1
    }
}
```

Property	Default	Description
maxDegreeOfParallelism	8 × core count	Max concurrent invocations across all blob-triggered functions in the app. Min: 1.
poisonBlobThreshold	5	Times to try a blob before moving to poison queue. Min: 1.

functions/blobTIO.ts

```typescript
import { app, InvocationContext, output, input } from "@azure/functions";
import { IBindingInfo } from "../models/models.js";
import { myService } from "../services/myService.js";

const blobInput = input.storageBlob({
  path: process.env.BindingInfoFile,
  connection: 'AzureWebJobsStorage',
});

const blobOutput = output.storageBlob({
  path: process.env.BindingInfoFile,
  connection: 'AzureWebJobsStorage',
});

export async function blobTrigger(blobItem: Buffer, context: InvocationContext): Promise<void> {
  const LOG_SOURCE = "blobTrigger";

  try {
    // Get bindingInfo file from blob storage
    const bindingInfo = context.extraInputs.get(blobInput) as IBindingInfo;

    // Do something with the blobItem
    const success = myService.ProcessBlobItem(blobItem);

    // Update bindingInfo and write to blob storage
    bindingInfo.lastRun = new Date();
    context.extraOutputs.set(blobOutput, bindingInfo);
  } catch (err) {
    console.error(LOG_SOURCE, "Blob item was not processed.", err);
    throw err;
  }
};

app.storageBlob('blobTrigger', {
  path: process.env.MyBlobPath,
  connection: 'AzureWebJobsStorage',
  extraInputs: [blobInput],
  extraOutputs: [blobOutput],
  handler: blobTrigger
});
```

Queue Storage Bindings



SYMPRAXIS
CONSULTING

Queue Storage: hosts.json configuration

```
"extensions": {
    "queues": {
        "maxPollingInterval": "00:00:02",
        "visibilityTimeout": "00:00:30",
        "batchSize": 16,
        "maxDequeueCount": 5,
        "newBatchThreshold": 8,
        "messageEncoding": "base64"
    }
}
```

Property	Default	Description
maxPollingInterval	00:01:00	Max interval between queue polls. Min: 100ms. Backs off incrementally up to this value.
visibilityTimeout	00:00:00	How long to hide a message after a failed attempt before retrying.
batchSize	16	Messages fetched and processed in parallel (max: 32). Set to 1 to disable parallelism on one instance.

Queue Storage: hosts.json configuration

```
"extensions": {
    "queues": {
        "maxPollingInterval": "00:00:02",
        "visibilityTimeout": "00:00:30",
        "batchSize": 16,
        "maxDequeueCount": 5,
        "newBatchThreshold": 8,
        "messageEncoding": "base64"
    }
}
```

Property	Default	Description
maxDequeueCount	5	Times to try a message before moving to <queue>-poison.
newBatchThreshold	N × batchSize / 2	When in-flight count drops to this, fetch another batch. N=vCPUs (1 on Consumption).
messageEncoding	base64	base64 or none. Controls how message content is encoded.

Hosts.json – Blob, Queue, Service Bus

```
{
    "concurrency": {
        "dynamicConcurrencyEnabled": true,
        "snapshotPersistenceEnabled": true
    }
}
```

- Alternative to specific configuration.
- dynamicConcurrencyEnabled ignores batchSize, newBatchThreshold for Queue and Blob triggers.

functions/queueOut.ts

```ts
import { app, HttpRequest, HttpResponseInit, InvocationContext, output } from "@azure/functions";
import { IMyQueueItem } from "../models/models.js";

const queueOutput = output.storageQueue({
  queueName: process.env.MyQueuePath,
  connection: 'AzureWebJobsStorage',
});

export async function httpRequest(request: HttpRequest, context: InvocationContext): Promise<HttpResponseInit> {
  const LOG_SOURCE = "queueTrigger";
  try {
    var requestBody = await request.json();
    var data: IMyQueueItem = requestBody as IMyQueueItem;
    context.extraOutputs.set(queueOutput, JSON.stringify(data));
    return { status: 200 };
  } catch (err) {
    console.error(LOG_SOURCE, "Failed http request", err);
    return { status: 500, body: JSON.stringify({ message: `Critical error - ${err}` }) };
  }
};

app.http('httpRequest', {
  methods: ['POST'],
  authLevel: 'anonymous',
  extraOutputs: [queueOutput],
  handler: httpRequest,
});
```

functions/queueTrigger.ts

```ts
import { app, InvocationContext } from "@azure/functions";
import { QueueClient } from "@azure/storage-queue";
import { IMyQueueItem } from "../models/models";
import { myService } from "../services/myService";

export async function queueTrigger(queueItem: IMyQueueItem, context: InvocationContext): Promise<void> {
  const LOG_SOURCE = "queueTrigger";

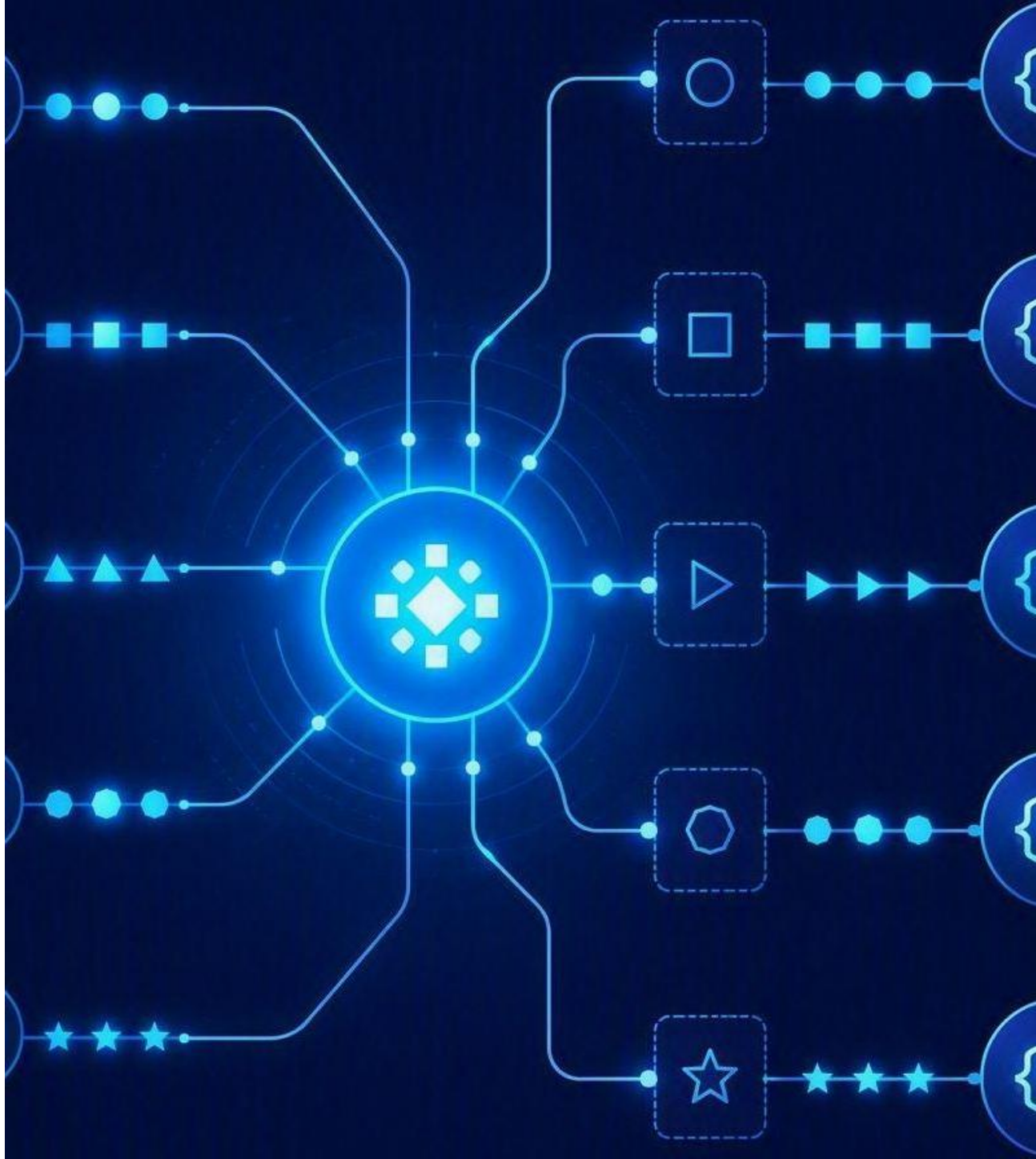
  try {
    // process queueItem
    const success = myService.ProcessQueueItem(queueItem);

    // Alternate: Requeue logic
    if (!success) {
      const queueClient = new QueueClient(
        process.env.AzureWebJobsStorage,
        process.env.MyQueuePath
      );

      const response = await queueClient.updateMessage(
        context.triggerMetadata.id as string,
        context.triggerMetadata.popReceipt as string,
        Buffer.from(JSON.stringify(queueItem)).toString("base64"),
        300
      );
    } else {
      context.log(LOG_SOURCE, "Queue item processed successfully.");
    }
  } catch (err) {
    console.error(LOG_SOURCE, "Queue item was not processed.", err);
    throw err;
  }
}

app.storageQueue('queueTrigger', {
  queueName: process.env.MyQueuePath,
  connection: 'AzureWebJobsStorage',
  handler: queueTrigger
});
```

Event Grid Bindings



SYMPRAXIS CONSULTING

Queues vs Event Grid

	Storage/Service Bus Queue	Event Grid Topic
Delivery	Pull — a consumer dequeues each item	Push — Event Grid delivers to every subscriber's endpoint
Consumers	One logical consumer (or a pool competing for the same items)	Many independent subscribers, each gets its own copy
Coupling	Publisher usually knows there's "a queue" downstream	Publisher doesn't know or care who's listening
Filtering	None built in — consumer inspects everything it dequeues	Per-subscription filters (event type, subject prefix/suffix, custom fields) — each subscriber only receives what it asked for
Durability model	Designed to buffer/hold a backlog while a slow consumer catches up	Retries for a window (default 24h) then dead-letters — not meant as a long-term backlog
Best for	"Do this piece of work reliably, exactly once, at whatever pace the worker can handle"	"Announce that X happened, let anyone interested react"

Event Grid: hosts.json configuration

```
"extensions": {
    "http": {
        "routePrefix": "api",
        "maxOutstandingRequests": 200,
        "maxConcurrentRequests": 100,
        "dynamicThrottlesEnabled": true
    }
}
```

- Custom Headers
- hsts – http strict transport security

Property	Default	Description
routePrefix	"api"	Route prefix for all HTTP/Event Grid endpoints. Set to "" to remove.
maxOutstandingRequests	200 (Consumption) / -1 (Premium+)	Max queued + in-flight requests before returning 429 Too Busy.
maxConcurrentRequests	100 (Consumption) / -1 (Premium+)	Max parallel Event Grid function executions.
dynamicThrottlesEnabled	true (Consumption) / false (Premium+)	Auto-throttle when system resources (CPU/memory/connections) exceed ~80%.

functions/eventGridTO.ts

```
1  import { app, EventGridEvent, InvocationContext, output } from "@azure/functions";
2
3  const eventGridOutput = output.eventGrid({
4    topicEndpointUri: 'ForwardTopicEndpointUri',
5    topicKeySetting: 'ForwardTopicKey',
6  });
7
8  const queueOutput = output.storageQueue({
9    queueName: process.env.MyQueuePath,
10   connection: 'AzureWebJobsStorage',
11 });
12
13 export async function eventGridListChanged(event: EventGridEvent, context: InvocationContext): Promise<void> {
14   const LOG_SOURCE = "eventGridListChanged";
15
16   try {
17     if (event.data.resource === "Groups/cf653542-be15-4568-87d4-37613d8a21f5") {
18       context.extraOutputs.set(queueOutput, JSON.stringify(event.data));
19     } else {
20       // Forward the received event as-is to the downstream custom topic
21       context.extraOutputs.set(eventGridOutput, event);
22     }
23   } catch (err) {
24     console.error(LOG_SOURCE, "General Failure", err);
25     throw err;
26   }
27 }
28
29 app.eventGrid('eventGridListChanged', {
30   handler: eventGridListChanged,
31   extraOutputs: [eventGridOutput, queueOutput]
32 });
```

Azure SQL Bindings



SYMPRAXIS
CONSULTING

Azure SQL Binding Security

- Input/Output binding requires Azure Function to have db_datareader/db_datawriter permissions
- Trigger binding requires additional permissions
  - Grant create table
  - Grant create schema
  - Grant select on target table
  - Grant view change tracking on target table
  - Create a schema for the function
  - Grant Alter on schema
  - Grant Select, Insert, Update, and Delete on schema

SQL Trigger Requirements

- Process creates it own tracking infrastructure
- CHANGE_RETENTION controls how far back change history is kept
- AUTO_CLEANUP
  can be turned off temporarily if you need to test without removing history

```
ALTER DATABASE [YourDatabaseName]
SET CHANGE_TRACKING = ON
(CHANGE_RETENTION = 2 DAYS,
AUTO_CLEANUP = ON);


ALTER TABLE [dbo].[JobStatus]
ENABLE CHANGE_TRACKING;
```

SQL Trigger: hosts.json configuration

```
"extensions": {
    "Sql": {
        "MaxBatchSize": 300,
        "PollingIntervalMs": 1000,
        "MaxChangesPerWorker": 100
    }
}
```

Property	Default	Description
MaxBatchSize	100	The maximum number of changes processed with each iteration of the trigger loop before being sent to the triggered function.
PollingIntervalMs	1000	The delay in milliseconds between processing each batch of changes.
MaxChangesPerWorker	1000	The upper limit on the number of pending changes in the user table that are allowed per application-worker.

functions/sqlTrigger.ts

```typescript
import { app, InvocationContext, SqlChange } from "@azure/functions";
import { IJobStatus } from "../models/models.js";
import { myService } from "../services/myService.js";

export async function sqlTrigger(changes: SqlChange<IJobStatus>[], context: InvocationContext): Promise<void> {
  const LOG_SOURCE = "sqlTrigger";

  try {
    for (const change of changes) {
      const success = myService.ProcessSqlChange(change.Item, change.Operation);

      if (!success) {
        context.log(LOG_SOURCE, `Row change was not processed. Operation: ${change.Operation}`);
      }
    }
  } catch (err) {
    console.error(LOG_SOURCE, "General Failure", err);
    throw err;
  }
}

app.sql('sqlTrigger', {
  tableName: 'dbo.JobStatus',
  connectionStringSetting: 'SqlConnectionString',
  handler: sqlTrigger
});
```

functions/sqlIO.ts

```
3  const sqlUpdateToken = input.sql({
4    commandText: 'dbo.sproc_UpdateLastToken',
5    commandType: 'StoredProcedure',
6    parameters: '@LastToken={LastToken}',
7    connectionStringSetting: 'SqlConnectionString',
8  });
9
10 const updateSPSiteUrl = input.sql({
11   commandText: 'dbo.sproc_UpdateSPSiteUrl',
12   commandType: 'StoredProcedure',
13   parameters: '@JobFamilyName={Name},@SPSiteUrl={Url}',
14   connectionStringSetting: 'SqlConnectionString',
15 });
16
17 // output.sql writes/upserts row(s) directly into a table (merge on the table's primary key),
18 // unlike input.sql above, which is used here to invoke stored procedures instead.
19 const jobStatusOutput = output.sql({
20   commandText: 'dbo.JobStatus',
21   connectionStringSetting: 'SqlConnectionString',
22 });
23
24 export async function sqlUpdateQueue(queueItem: { LastToken: string, Name: string, Url: string; }, context:
   InvocationContext): Promise<void> {
25   const LOG_SOURCE = "sqlUpdateQueue";
26
27   try {
28     if (queueItem.LastToken?.length > 0) {
29       const tokenUpdate = context.extraInputs.get(sqlUpdateToken);
30     }
31     if (queueItem.Name?.length > 0 && queueItem.Url?.length > 0) {
32       const sqlUpdate = context.extraInputs.get(updateSPSiteUrl);
33     }
34
35     // Record/refresh a status row for this job family - upserted by dbo.JobStatus's primary key (JobFamilyName)
36     context.extraOutputs.set(jobStatusOutput, {
37       JobFamilyName: queueItem.Name,
38       SPSiteUrl: queueItem.Url,
39       LastToken: queueItem.LastToken,
40       LastUpdated: new Date().toISOString(),
41     });
42   } catch (err) {
43     console.error(LOG_SOURCE, "General Failure", err);
44     throw err;
45   }
46 }
47
48 app.storageQueue('sqlUpdateQueue', {
49   queueName: process.env.SqlUpdateQueueName,
50   connection: 'AzureWebJobsStorage',
51   extraInputs: [sqlUpdateToken, updateSPSiteUrl],
52   extraOutputs: [jobStatusOutput],
53   handler: sqlUpdateQueue
54 });
```

functions/sqlIO.ts

```
const sqlUpdateToken = input.sql({
    commandText: 'dbo.sproc_UpdateLastToken',
    commandType: 'StoredProcedure',
    parameters: '@LastToken={LastToken}',
    connectionStringSetting: 'SqlConnectionString',
});

const updateSPSiteUrl = input.sql({
    commandText: 'dbo.sproc_UpdateSPSiteUrl',
    commandType: 'StoredProcedure',
    parameters: '@JobFamilyName={Name},@SPSiteUrl={Url}',
    connectionStringSetting: 'SqlConnectionString',
});

// output.sql writes/upserts row(s) directly into a table (merge on the table's primary key),
// unlike input.sql above, which is used here to invoke stored procedures instead.
const jobStatusOutput = output.sql({
    commandText: 'dbo.JobStatus',
    connectionStringSetting: 'SqlConnectionString',
});

export async function sqlUpdateQueue(queueItem: { LastToken: string, Name: string, Url: string; }, context:
InvocationContext): Promise<void> {
    const LOG_SOURCE = "sqlUpdateQueue";

```

functions/sqlIO.ts

```
24  export async function sqlUpdateQueue(queueItem: { LastToken: string, Name: string, Url: string; }, context:
    InvocationContext): Promise<void> {
25    const LOG_SOURCE = "sqlUpdateQueue";
26
27    try {
28      if (queueItem.LastToken?.length > 0) {
          const tokenUpdate = context.extraInputs.get(sqlUpdateToken);
30      }
31      if (queueItem.Name?.length > 0 && queueItem.Url?.length > 0) {
          const sqlUpdate = context.extraInputs.get(updateSPSiteUrl);
33      }
34
35      // Record/refresh a status row for this job family - upserted by dbo.JobStatus's primary key (JobFamilyName)
        context.extraOutputs.set(jobStatusOutput, {
37        JobFamilyName: queueItem.Name,
38        SPSiteUrl: queueItem.Url,
39        LastToken: queueItem.LastToken,
40        LastUpdated: new Date().toISOString(),
41      });
42    } catch (err) {
43      console.error(LOG_SOURCE, "General Failure", err);
44      throw err;
45    }
46  }
47
48  app.storageQueue('sqlUpdateQueue', {
49    queueName: process.env.SqlUpdateQueueName,
50    connection: 'AzureWebJobsStorage',
51    extraInputs: [sqlUpdateToken, updateSPSiteUrl],
      extraOutputs: [jobStatusOutput],
53    handler: sqlUpdateQueue
54  });
```

Resources

- Azure SQL trigger for Functions